

All for one and none for all: Compiling polymorphic relations without monomorphization

Dmitri Volkov, Yafei Yang, Chung-chieh Shan

2026 August 24

All for one and none forall

Represent all concrete
instances of a
polymorphic relation

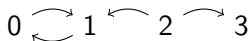
using only a single
concrete instance

to compile away type
variables

Introducing semiringKanren

Polymorphic semiringKanren

Representing relations



```

(defrel (graph
  (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
  
```

		to			
		0	1	2	3
from	0	0	1	0	0
	1	1	0	0	0
	2	0	1	0	1
	3	0	0	0	0

Represent graphs as relations or as adjacency matrices.

semiringKanren computes matrices for arbitrary relations!

disj is addition

```
(defrel (r0 (x : (Num 4)) (y : (Num 4)))  
  (disj  
    (== x 0)  
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

conj is multiplication

```
(defrel (r1 (x : (Num 4)) (y : (Num 4)))  
  (disj  
    (== x 0)  
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

fresh is summation

Vertices with incoming edges.



```

(defrel (incoming-edge-vertices (y : (Num 4)))
  (fresh (x : (Num 4))
    (graph x y)))

```

$$\sum_x \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 & 1 \end{bmatrix}$$

Types

semiringKanren values inhabit finite algebraic data types.

		left	right
(left 0) :	(Sum (Num 2) (Num 3))	$\begin{bmatrix} 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$
(left 1) :	(Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$
(right 0) :	(Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$
(right 1) :	(Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$
(right 2) :	(Sum (Num 2) (Num 3))	$\begin{bmatrix} 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$

(semiringKanren also supports unit and product types)

Representing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Representing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))  
  (disj  
    (conj (== from 0) (== to 1))  
    (conj (== from 1) (== to 0))  
    (conj (== from 2) (== to 1))  
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Representing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Representing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Representing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3)))))
```

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Representing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))  
  (disj  
    (conj (== from 0) (== to 1))  
    (conj (== from 1) (== to 0))  
    (conj (== from 2) (== to 1))  
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$



Introducing semiringKanren

Polymorphic semiringKanren

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{array}{c}
 \text{left} \left\{ \begin{array}{ccc|ccc}
 0 & 0 & 0 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0 & 1 \\
 1 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1 & 0 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & 0 & 0
 \end{array} \right.
 \end{array}$$

right

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{array}{c}
 \text{left} \\
 \left[\begin{array}{ccc|ccc}
 0 & 0 & 0 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0 & 1 \\
 \hline
 1 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1 & 0 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & 0 & 0
 \end{array} \right]
 \end{array}$$

right

Larger sum swap

```

(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))

```

$$\begin{bmatrix}
 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & 0 & 0 & 0 \\
 0 & 0 & 0 & 1 & 0 & 0 & 0
 \end{bmatrix}$$

Larger sum swap

```

(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a)))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))

```

Diagram illustrating the sum swap operation using a 7x7 matrix. The matrix is partitioned into two regions: **left** (rows 1-4, columns 1-4) and **right** (rows 1-4, columns 5-7). The **right** region is highlighted with an orange box.

					1	0	0
					0	1	0
					0	0	1
1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0

Larger sum swap

```

(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))

```

	left						
	0	0	0	0	1	0	0
	0	0	0	0	0	1	0
	0	0	0	0	0	0	1
right	1	0	0	0	0	0	0
	0	1	0	0	0	0	0
	0	0	1	0	0	0	0
	0	0	0	1	0	0	0

Polymorphic sum swap

```
(defrel (sum-swap
  (x : (Sum  $\alpha$   $\beta$ ))
  (y : (Sum  $\beta$   $\alpha$ )))
  (disj
    (fresh (a :  $\alpha$ )
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b :  $\beta$ )
      (conj
        (== x (right b))
        (== y (left b))))))
```

[?]

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{array}{ccc}
 \left. \begin{array}{l} x \mapsto (\text{right } \underline{2_3}) \\ y \mapsto (\text{left } \underline{2_3}) \end{array} \right\} 2_3 = 2_3 & & \left. \begin{array}{l} x \mapsto (\text{right } \underline{3_4}) \\ y \mapsto (\text{left } \underline{3_4}) \end{array} \right\} 3_4 = 3_4 \\
 \swarrow \quad \searrow & & \swarrow \quad \searrow \\
 x \text{ is } (\text{right } \dots) & y \text{ is } (\text{left } \dots) & \beta\text{-typed parts equal}
 \end{array}$$